

1 Formalized TypeSpec and Elixir Type

Annotations	A	$::=$	$\overline{\text{@spec } E} \mid \varepsilon$
Expressions	E	$::=$	$x(\overline{T}) :: T \mid x(\overline{T}) :: T \text{ when } \overline{\alpha : T}$
Types	T	$::=$	$\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \text{tuple}() \mid \{\overline{T}\} \mid \% \{\overline{F}\}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \llcorner \mid \llcorner :: n \llcorner \mid \llcorner :: n * n \llcorner \mid \llcorner :: n, _ :: n * n \llcorner$ $\mid (\overline{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F	$::=$	$\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

(where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names)

Figure 1: TypeSpec

Annotations	A	$::=$	$\$S \mid \$S \text{ when } \overline{\alpha : T}$
Signatures	S	$::=$	$\overline{T} \rightarrow T \mid S \text{ and } S$
Types	T	$::=$	$\alpha \mid n..n \mid T \text{ or } T \mid T \text{ and } T \mid \text{not } T$ $\mid K \mid \text{term}() \mid \text{none}() \mid \text{dynamic}()$ $\mid \{\overline{T}\} \mid \{\overline{T}, \dots\} \mid \% \{\overline{F}\} \mid \% \{\dots, \overline{F}\}$ $\mid (\overline{T} \rightarrow T) \mid \text{non_empty_list}(T, T)$
Key types	K	$::=$	$\text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}() \mid \text{binary}()$ $\mid \text{tuple}() \mid \text{open_map}() \mid \text{fun}()$ $\mid \text{empty_list}() \mid \text{non_empty_list}(\text{term}(), \text{empty_list}())$
Fields	F	$::=$	$:k \Rightarrow T \mid K \Rightarrow \text{if_set}(T)$

(where x ranges over expression variables, α ranges over type variables, $n..n$ ranges over integers while invisible to the users, and k ranges over atom names)

Figure 2: Elixir Type

2 TypeSpec Built-in Types and Syntactic Sugars

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::_8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::_1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:_::struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

(built-in types with prepending asterisk (*) are only listed for documentation)

Table 1: TypeSpec Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k: T	:k => T
%k{F}	%{:__struct__ => :k, F}

Table 2: TypeSpec Syntactic Sugars

3 Elixir Type Built-in Types and Syntactic Sugars

Built-in types	Defined as
non_empty_list(T)	non_empty_list(T, empty_list())
list(T)	empty_list() or non_empty_list(T)
list()	list(term())
improper_list()	non_empty_list(term(), not list())

Table 3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
{...}	tuple()
%{...}	open_map()
k: T	:k => T
K => not_set()	K => if_set(none())

Table 4: Elixir Type Syntactic Sugars

4 Translation from TypeSpec to Elixir Type

$\llbracket \cdot \rrbracket: \text{TypeSpec} \rightarrow \text{Elixir type}$

Figure 3: Translation

4.1 Translation in Annotations

$$\llbracket \text{@spec } \bar{E} \rrbracket = \begin{cases} \$ S \text{ when } \overline{\alpha'_n : T'_n} & \text{if } \exists n = \text{length}(\text{@spec } \bar{E}), \llbracket E_n \rrbracket \approx \llbracket x(\bar{T}_n) :: T_n \text{ when } \overline{\alpha_n : T_n} \rrbracket \\ & \text{where } [\alpha_n/\alpha'_n], [T_n/T'_n] \text{ iff } \exists 0 \leq i < n, \alpha_i = \alpha_n \\ \$ S \text{ when } \overline{\alpha_n : T_n} & \text{if } \exists n = \text{length}(\text{@spec } \bar{E}), \llbracket E_n \rrbracket \approx \llbracket x(\bar{T}_n) :: T_n \text{ when } \overline{\alpha_n : T_n} \rrbracket \\ \$ S & \text{otherwise} \end{cases}$$

$$\llbracket \epsilon \rrbracket =$$

Figure 4: Translation in Annotations

4.2 Translation in Types

```

 $\llbracket \alpha \rrbracket = \alpha$ 
 $\llbracket T_1 | T_2 \rrbracket = \llbracket T_1 \rrbracket \text{ or } \llbracket T_2 \rrbracket$ 
 $\llbracket \text{any}() \rrbracket = \text{term}()$ 
 $\llbracket \text{none}() \rrbracket = \text{none}()$ 
 $\llbracket \text{atom}() \rrbracket = \text{atom}()$ 
 $\llbracket :k \rrbracket = :k$ 
 $\llbracket \text{pid}() \rrbracket = \text{pid}()$ 
 $\llbracket \text{port}() \rrbracket = \text{port}()$ 
 $\llbracket \text{reference}() \rrbracket = \text{reference}()$ 
 $\llbracket \text{float}() \rrbracket = \text{float}()$ 
 $\llbracket \text{integer}() \rrbracket = \text{integer}()$ 
 $\llbracket \text{neg\_integer} \rrbracket = -\text{int}--1$ 
 $\llbracket \text{non\_neg\_integer} \rrbracket = 0--\text{inf}$ 
 $\llbracket \text{pos\_integer} \rrbracket = 1--\text{inf}$ 
 $\llbracket \text{tuple}() \rrbracket = \text{tuple}()$ 
 $\llbracket \{\bar{T}\} \rrbracket = \{\llbracket \bar{T} \rrbracket\}$ 
 $\llbracket \% \{\bar{F}\} \rrbracket = \% \{\llbracket \bar{F} \rrbracket\}$ 
 $\llbracket [] \rrbracket = \text{empty\_list}()$ 
 $\llbracket \text{nonempty\_maybe\_improper\_list}(T_1, T_2) \rrbracket = \text{non\_empty\_list}(\llbracket T_1 \rrbracket, \llbracket T_2 \rrbracket)$ 
 $\llbracket \llbracket \rrbracket \rrbracket = \text{undefined}$ 
 $\llbracket \llbracket \_ :: n \rrbracket \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$ 
 $\llbracket \llbracket \_ :: \_ * n \rrbracket \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$ 
 $\llbracket \llbracket \_ :: n, \_ :: \_ * n \rrbracket \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$ 
 $\llbracket (\bar{T} \rightarrow T) \rrbracket = (\llbracket \bar{T} \rrbracket \rightarrow \llbracket T \rrbracket)$ 
 $\llbracket (\dots \rightarrow T) \rrbracket = \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ \text{undefined} & \text{otherwise} \end{cases}$ 
 $\llbracket n \rrbracket = n--n$ 
 $\llbracket n..n' \rrbracket = n--n'$ 

```

Figure 5: Translation in Types

4.3 Translation in Fields

$$\begin{aligned} \llbracket \text{required}(\tau_1) \Rightarrow \tau_2 \rrbracket &= \begin{cases} :k \Rightarrow \llbracket \tau_2 \rrbracket & \text{if } \llbracket \tau_1 \rrbracket \approx :k \\ \text{undefined} & \text{otherwise} \end{cases} \\ \llbracket \text{optional}(\tau_1) \Rightarrow \tau_2 \rrbracket &= \begin{cases} :k \Rightarrow \llbracket \tau_2 \rrbracket & \text{if } \llbracket \tau_1 \rrbracket \approx :k \\ K \Rightarrow \text{if_set}(\llbracket \tau_2 \rrbracket) & \text{if } \llbracket \tau_1 \rrbracket \approx K \\ \text{undefined} & \text{otherwise} \end{cases} \end{aligned}$$

Idea :

$$\llbracket \text{required}(\tau_1) \Rightarrow \tau_2 \rrbracket = \begin{cases} :k \Rightarrow \llbracket \tau_2 \rrbracket & \text{if } \llbracket \tau_1 \rrbracket \approx :k \\ \text{undefined} & \text{otherwise} \end{cases}$$

Figure 6: Translation in Fields